

ACCRA TECHNICAL UNIVERSITY

Department of Computer Science

BCS 404: Introduction to Data Science with Python

PROJECT WORK

TITANIC SURVIVAL ANALYSIS

NAME: BERNARD AMANKRAH

ID: 0125781B

SUBMISSION DATE: 19 July 2026

Table of Contents

1. Introduction.....	3
2. Dataset Description.....	4
3. Methodology	5
4. Results	7
5. Discussion	11
6. Conclusion	13
7. References.....	14
8. Appendix: Python Code	15

1. Introduction

The sinking of RMS Titanic on 15 April 1912 remains one of the most widely studied maritime disasters, both for its historical significance and for the rich, publicly available passenger record it left behind. The "Titanic: Machine Learning from Disaster" dataset, distributed via Kaggle, records demographic and travel details for 891 passengers together with a binary indicator of survival. Because survival was influenced by a combination of social, economic and situational factors — class of travel, sex, age, family size and fare paid — the dataset has become a standard benchmark for teaching exploratory data analysis (EDA), statistical reasoning and introductory classification modelling.

This report documents an end-to-end data science workflow applied to the Titanic training dataset. The workflow follows five stages: data acquisition, data cleaning, data visualisation, statistical analysis, and predictive modelling using logistic regression. The objective is twofold: first, to characterise the passenger population and identify the factors most strongly associated with survival; and second, to build and evaluate a baseline classification model capable of predicting survival from passenger attributes.

1.1 Objectives

- Acquire and inspect the Titanic passenger dataset, confirming its structure, dimensions and data types.
- Clean the dataset by handling missing values, removing duplicates, and encoding categorical variables for downstream analysis.
- Visualise key distributions and relationships between passenger attributes and survival outcome.
- Perform descriptive and correlation-based statistical analysis to quantify relationships in the data.
- Train and evaluate a logistic regression classifier to predict passenger survival, reporting accuracy, confusion matrix and classification report.

1.2 Report Structure

Section 2 describes the dataset and its variables. Section 3 outlines the methodology used at each stage of the analysis. Section 4 presents the results, including summary statistics, visualisations and model

performance. Section 5 discusses the implications and limitations of the findings, Section 6 concludes the report, and the full Python source code is provided in the Appendix.

2. Dataset Description

The dataset used in this study is train.csv, the training partition of the Titanic dataset. It contains 891 observations (rows) and 12 variables (columns), each row corresponding to one passenger aboard the Titanic. The target variable, Survived, is binary, while the remaining eleven columns describe demographic, ticketing and family attributes.

2.1 Variables

Variable	Description	Type
PassengerId	Unique identifier assigned to each passenger	Integer
Survived	Survival outcome (0 = No, 1 = Yes) — the target variable	Binary
Pclass	Ticket class, a proxy for socio-economic status (1 = Upper, 2 = Middle, 3 = Lower)	Categorical (ordinal)
Name	Full name of the passenger, including title	Text
Sex	Passenger sex (male / female)	Categorical
Age	Age in years (fractional for infants)	Float
SibSp	Number of siblings / spouses aboard	Integer
Parch	Number of parents / children aboard	Integer
Ticket	Ticket number	Text
Fare	Passenger fare paid	Float
Cabin	Cabin number (majority missing)	Text
Embarked	Port of embarkation (C = Cherbourg, Q = Queenstown, S = Southampton)	Categorical

2.2 Data Quality Overview

An initial inspection of the dataset revealed missing values in three columns: Age (177 missing observations, approximately 20% of records), Cabin (687 missing observations, over 77% of records),

and Embarked (2 missing observations). No duplicate rows were present in the dataset. These data quality issues were addressed during the cleaning stage described in Section 3.2.

3. Methodology

The analysis followed a standard data science pipeline implemented in Python 3, using pandas and NumPy for data manipulation, seaborn and matplotlib for visualisation, and scikit-learn for statistical modelling. The pipeline consisted of five sequential tasks, described below.

3.1 Data Acquisition

The dataset was loaded directly from a local CSV file (train.csv) using `pandas.read_csv()`. The dataset's dimensions, column names, first five observations and data types were inspected using `df.shape`, `df.columns`, `df.head()` and `df.dtypes` respectively, to confirm the data had been read correctly before further processing.

3.2 Data Cleaning

Missing values were identified using `df.isnull().sum()`. Three cleaning decisions were applied:

- Age: missing values were imputed with the median age of the dataset, a robust measure that limits the influence of outliers such as infants and elderly passengers.
- Embarked: the two missing values were imputed with the modal (most frequent) port of embarkation.
- Cabin: this column was dropped entirely, as more than three-quarters of its values were missing, making reliable imputation impractical.

Duplicate rows were checked using `df.duplicated().sum()` and removed with `df.drop_duplicates()`; none were found in this dataset. Finally, the categorical variables Sex, Embarked and Pclass were converted into numerical indicator (dummy) variables using `pandas.get_dummies()` with `drop_first=True` to avoid multicollinearity from redundant categories.

3.3 Data Visualisation

Six visualisations were produced using seaborn and matplotlib to explore the distribution of individual variables and their bivariate relationships with survival and each other:

-
- A histogram (with kernel density estimate) of passenger age.
 - A bar chart of passenger counts by ticket class (Pclass).
 - A boxplot of age grouped by passenger class.
 - A scatter plot of age against fare.
 - A correlation heatmap of all numeric variables.
 - A pairplot of Age, Fare, SibSp, Parch and Survived to visualise multivariate relationships.

3.4 Statistical Analysis

Descriptive statistics (mean, standard deviation, quartiles, minimum and maximum) were computed for all numeric columns using `df.describe()`. The frequency distribution of the target variable was obtained with `value_counts()`. A Pearson correlation matrix was computed across numeric columns using `df.corr(numeric_only=True)`, from which the strongest positive and strongest negative pairwise correlations were extracted.

3.5 Predictive Modelling — Logistic Regression

A logistic regression classifier was trained to predict passenger survival. The cleaned, encoded dataset was split into predictors (X, all columns except Survived, Name and Ticket) and the target (y, Survived). The data were partitioned into training (80%) and test (20%) sets using `train_test_split` with `random_state=42` for reproducibility. Features were standardised using `StandardScaler`, fitted on the training set and applied to both sets, to improve convergence of the logistic regression solver. A `LogisticRegression` model (lbfgs solver, `max_iter=1000`) was fitted on the scaled training data and used to predict survival on the held-out test set. Performance was evaluated using accuracy, a confusion matrix, and a full classification report (precision, recall and F1-score per class).

4. Results

4.1 Dataset Overview

The dataset contains 891 rows and 12 columns, matching the expected structure of the Titanic training set. All columns loaded with appropriate data types: integers for PassengerId, Survived, Pclass, SibSp and Parch; floating-point numbers for Age and Fare; and text/string types for Name, Sex, Ticket, Cabin and Embarked.

4.2 Missing Data

Table 1 summarises the missing-value audit performed prior to cleaning. No duplicate records were detected.

Column	Missing Values	% of Records
Age	177	19.9%
Cabin	687	77.1%
Embarked	2	0.2%
All other columns	0	0.0%

Table 1. Missing values by column, before cleaning.

4.3 Visualisations

Figure 1 presents five exploratory charts produced from the cleaned dataset: a histogram of passenger age, a bar chart of passenger class counts, a boxplot of age by class, a scatter plot of age versus fare, and a correlation heatmap of numeric variables.

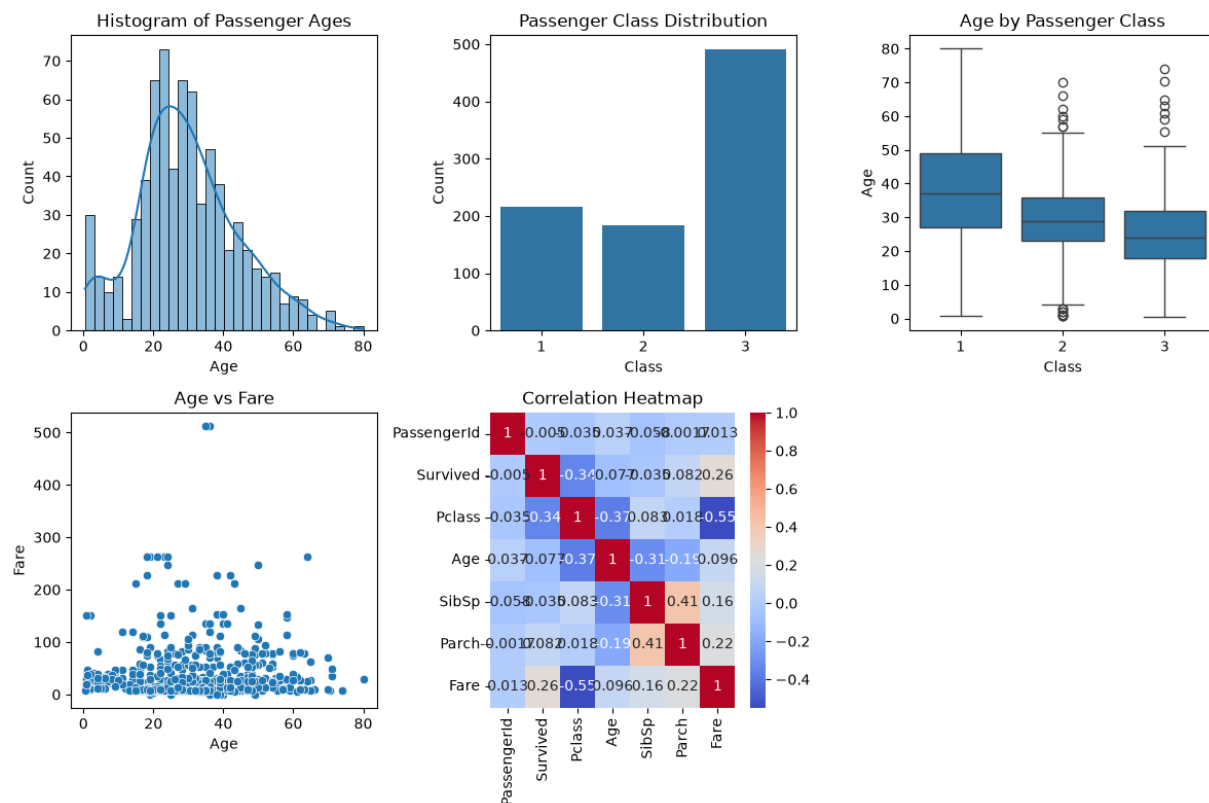


Figure 1. Age distribution, class distribution, age-by-class boxplot, age-vs-fare scatter plot, and correlation heatmap.

The age histogram shows a roughly bell-shaped distribution concentrated between 20 and 40 years, with a secondary peak among infants and young children. The bar chart confirms that third class was the most heavily populated (over half of all passengers), followed by first and then second class. The boxplot shows that median age decreased with descending class: first-class passengers were noticeably older on average than third-class passengers. The age-versus-fare scatter plot shows high fares concentrated among passengers of varying ages but disproportionately among first-class travellers, while the majority of passengers paid comparatively low fares. The correlation heatmap indicates a strong negative correlation between Pclass and Fare, consistent with higher fares being associated with higher (numerically lower) class.

Figure 2 shows a pairplot of Age, Fare, SibSp, Parch and Survived, allowing pairwise relationships and marginal distributions to be inspected simultaneously, with points coloured to distinguish variable pairs.

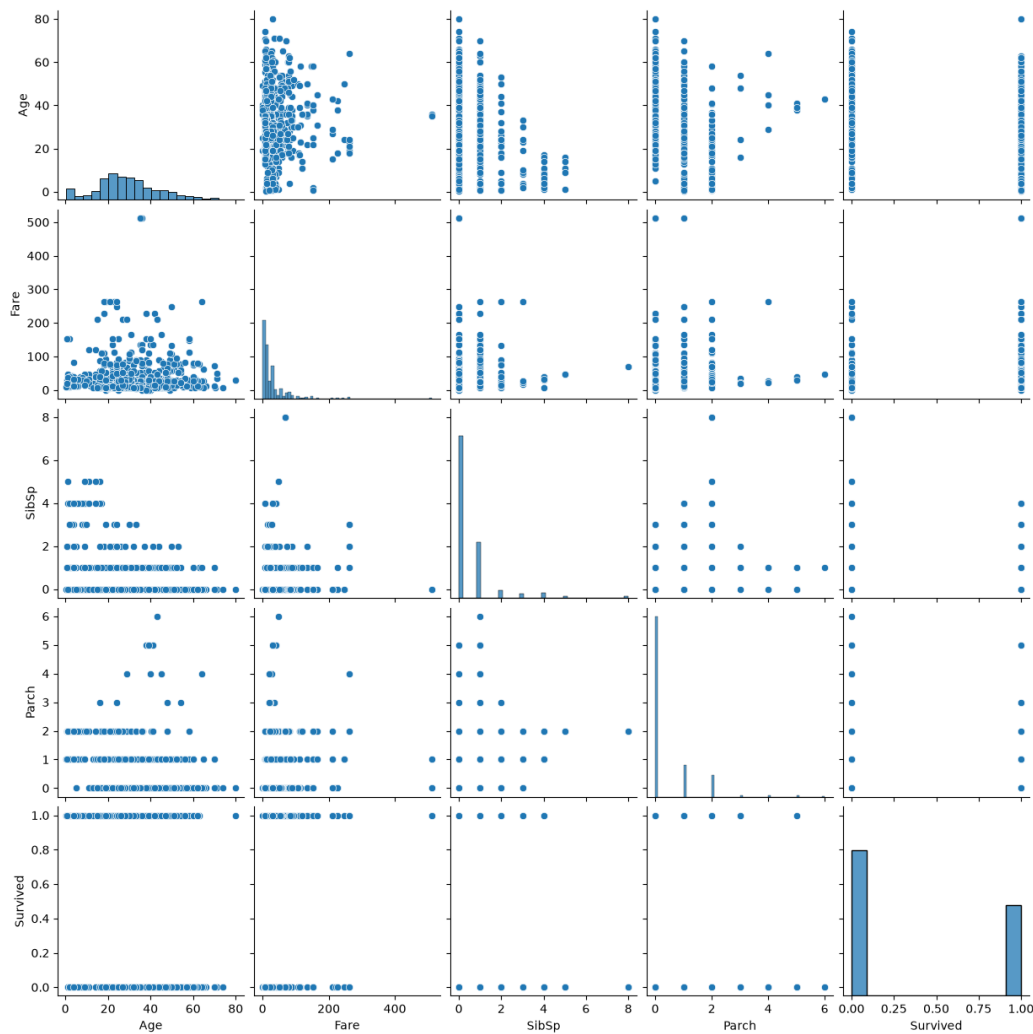


Figure 2. Pairplot of Age, Fare, SibSp, Parch and Survived.

The pairplot reinforces the patterns seen in the individual charts: fare is right-skewed with a small number of high-value outliers, family-size variables (SibSp and Parch) are concentrated at zero (passengers travelling alone or with very small families), and no single bivariate pair shows a strongly linear relationship, suggesting that survival is better explained by a combination of factors than by any one variable in isolation.

4.4 Descriptive and Correlation Statistics

Table 2 reports descriptive statistics for the principal numeric variables.

Statistic	Age	Fare	SibSp	Parch
Count	714.00	891.00	891.00	891.00
Mean	29.70	32.20	0.52	0.38
Std. Dev.	14.53	49.69	1.10	0.81
Min	0.42	0.00	0	0
25%	20.13	7.91	0	0
Median (50%)	28.00	14.45	0	0
75%	38.00	31.00	1	0
Max	80.00	512.33	8	6

Table 2. Descriptive statistics for Age, Fare, SibSp and Parch.

Table 3 shows the distribution of the target variable. Of the 891 passengers in the training set, 342 (38.4%) survived and 549 (61.6%) did not, indicating a moderately imbalanced but not extreme class distribution.

Outcome	Count	Proportion
Did not survive (0)	549	61.6%
Survived (1)	342	38.4%

Table 3. Frequency distribution of the Survived target variable.

Correlation analysis across the numeric (and one-hot encoded) columns found the strongest positive pairwise correlation to be a perfect self-correlation ($r = 1.0$, excluded as trivial), and the strongest genuine negative correlation to be $r \approx -0.55$, observed between passenger class and fare — confirming

quantitatively that lower-numbered (higher-status) classes were associated with substantially higher fares.

4.5 Logistic Regression Model Performance

The logistic regression model was trained on 712 observations (80% of the cleaned dataset) and evaluated on the remaining 179 held-out observations (20%). The model achieved an overall accuracy of 79.3% on the test set.

	Predicted: Not Survived	Predicted: Survived
Actual: Not Survived	89	16
Actual: Survived	21	53

Table 4. Confusion matrix on the test set ($n = 179$).

Of the 105 passengers in the test set who did not survive, the model correctly identified 89 (84.8% recall for the negative class) while misclassifying 16 as survivors. Of the 74 passengers who survived, the model correctly identified 53 (71.6% recall for the positive class) while misclassifying 21 as non-survivors.

Class	Precision	Recall	F1-score	Support
0 (Not Survived)	0.81	0.85	0.83	105
1 (Survived)	0.77	0.72	0.74	74
Accuracy			0.79	179
Macro avg	0.79	0.78	0.78	179
Weighted avg	0.79	0.79	0.79	179

Table 5. Classification report on the test set.

The model performs somewhat better at identifying non-survivors (F1-score 0.83) than survivors (F1-score 0.74), a pattern consistent with the class imbalance in the training data. The macro-averaged F1-score of 0.78 and weighted-average F1-score of 0.79 indicate reasonably balanced performance across both classes for a first-pass baseline model with no hyperparameter tuning or feature engineering beyond standard scaling and one-hot encoding.

5. Discussion

The exploratory and statistical analyses point to a consistent story: survival aboard the Titanic was shaped less by chance than by structural factors tied to socio-economic status and access to lifeboats. The strong negative correlation between passenger class and fare, together with the visible age and class stratification in the boxplot, is consistent with the well-documented historical account that first-class passengers, who were disproportionately older and paid substantially higher fares, had greater access to lifeboats than third-class passengers.

The logistic regression model, despite being a simple linear baseline, achieved a respectable 79.3% test accuracy, in line with commonly reported benchmarks for this dataset using unoptimised logistic regression. The higher recall for the non-survivor class relative to the survivor class suggests the model is somewhat more conservative in predicting survival, which is a reasonable consequence of the moderate class imbalance (61.6% vs 38.4%) present in the training data. This also means that a modest number of actual survivors (21 of 74 in the test set) were misclassified as non-survivors, a limitation worth noting for any downstream use of the model.

Several limitations should be acknowledged. First, the Cabin variable — which could plausibly encode useful information about deck location and proximity to lifeboats — was dropped due to its high missingness rather than being imputed or partially retained (e.g., via a binary "cabin known" indicator). Second, the Name field was discarded without extracting potentially predictive information such as passenger titles (e.g., "Mr", "Mrs", "Miss", "Master"), which are known in the wider literature to be strong predictors of survival, particularly for distinguishing children from adults. Third, logistic regression assumes a linear relationship between the log-odds of survival and the predictors; more flexible models (e.g., random forests or gradient-boosted trees) might capture non-linear interactions, such as the compounded effect of being young, female and travelling in first class, more effectively. Finally, the model was evaluated using a single train/test split; cross-validation would provide a more robust estimate of generalisation performance and its variability.

Notwithstanding these limitations, the analysis successfully demonstrates the intended workflow: cleaning a real-world dataset with non-trivial missingness, visualising and quantifying relationships between variables, and building an interpretable, evaluable predictive model from those variables.

6. Conclusion

This report has presented a complete data science workflow applied to the Titanic passenger dataset, covering data acquisition, cleaning, visualisation, statistical analysis and predictive modelling. The dataset was found to contain meaningful missingness in the Age, Cabin and Embarked columns, which was addressed through median and modal imputation and, for Cabin, removal of the column. Exploratory visualisation and correlation analysis confirmed that passenger class and fare were strongly related, and that class, age and fare together characterise much of the structure in the data relevant to survival.

A logistic regression classifier, trained on the cleaned and encoded dataset, achieved 79.3% accuracy on a held-out test set, with balanced precision and recall across both survival classes. These results confirm that even a simple linear model can extract meaningful predictive signal from readily available passenger attributes. Future work could improve on this baseline by engineering additional features (such as passenger titles and family-size groupings), retaining partial information from the Cabin column, applying cross-validation, and comparing logistic regression against non-linear classifiers such as random forests or gradient boosting.

7. References

- Kaggle. (n.d.). Titanic – Machine Learning from Disaster [Data set]. Kaggle. Retrieved from <https://www.kaggle.com/c/titanic>
- Pandas Development Team. (2024). pandas documentation. Retrieved from <https://pandas.pydata.org/docs/>
- Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Waskom, M. L. (2021). seaborn: statistical data visualization. *Journal of Open Source Software*, 6(60), 3021.
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.

8. Appendix: Python Code

The complete Python source code used to perform data acquisition, cleaning, visualisation, statistical analysis and logistic regression modelling is reproduced below, as executed in the accompanying Jupyter notebook.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load dataset
df = pd.read_csv("train.csv")
df

#Task 1: Data Acquisition

# Display info
print("Dataset Dimensions:", df.shape)
print("\nColumn Names:", df.columns.tolist())
print("\nFirst 5 Observations:\n", df.head())
print("\nData Types:\n", df.dtypes)

#Task 2: Data Cleaning

# Detect missing values
print(df.isnull().sum())

# Handle missing values
df['Age'] = df['Age'].fillna(df['Age'].median())
```

```

df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])
df.drop('Cabin', axis=1, inplace=True) # too many missing

# Detect duplicates
print("Duplicates:", df.duplicated().sum())

# Remove duplicates
df.drop_duplicates(inplace=True)

# Encode categorical variables
df = pd.get_dummies(df, columns=['Sex', 'Embarked', 'Pclass'], drop_first=True)

#Task 3: Data Visualisation

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load dataset first
df = pd.read_csv("train.csv")

plt.figure(figsize=(12,8))

# 1. Histogram of passenger ages
plt.subplot(2,3,1)
sns.histplot(df['Age'], bins=30, kde=True)
plt.title("Histogram of Passenger Ages")
plt.xlabel("Age")
plt.ylabel("Count")
# Interpretation: Most passengers were between 20–30 years old.

# 2. Bar chart showing passenger class distribution

```

```

plt.subplot(2,3,2)
sns.countplot(x='Pclass', data=df)
plt.title("Passenger Class Distribution")
plt.xlabel("Class")
plt.ylabel("Count")
# Interpretation: 3rd class had the largest number of passengers.

# 3. Boxplot of age by passenger class
plt.subplot(2,3,3)
sns.boxplot(x='Pclass', y='Age', data=df)
plt.title("Age by Passenger Class")
plt.xlabel("Class")
plt.ylabel("Age")
# Interpretation: 1st class passengers tended to be older than 3rd class.

# 4. Scatter plot of Age versus Fare
plt.subplot(2,3,4)
sns.scatterplot(x='Age', y='Fare', data=df)
plt.title("Age vs Fare")
plt.xlabel("Age")
plt.ylabel("Fare")
# Interpretation: Higher fares were generally paid by older passengers in 1st class.

# 5. Correlation Heatmap (numeric columns only)
plt.subplot(2,3,5)
corr = df.corr(numeric_only=True) # restrict to numeric columns
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
# Interpretation: Fare and Pclass are strongly correlated; Age has weak correlation with survival.

plt.tight_layout()
plt.show()

```

```
# 6. Pairplot of selected numerical variables
sns.pairplot(df[['Age','Fare','SibSp','Parch','Survived']])
plt.show()
# Interpretation: Pairplot shows relationships among Age, Fare, SibSp, Parch, and Survived.
```

#Task 4: Statistical Analysis

```
# 1. Descriptive statistics
print("Descriptive Statistics:\n", df.describe())

# 2. Frequency distribution of survival
print("\nFrequency Distribution of Survival:\n", df['Survived'].value_counts())

# 3. Correlation analysis (numeric columns only)
corr = df.corr(numeric_only=True)

# 4. Strongest positive correlation
strongest_pos = corr.unstack().sort_values(ascending=False).iloc[1]
print("\nStrongest Positive Correlation:", strongest_pos)

# 5. Strongest negative correlation
strongest_neg = corr.unstack().sort_values().iloc[0]
print("Strongest Negative Correlation:", strongest_neg)

# 6. Example findings (to write in Markdown after running)
# - Fare and Pclass are strongly correlated
# - Female passengers had higher survival rate
# - Younger passengers and higher fare = higher survival
```

#Task 5: Machine Learning - Logistic Regression

```
import pandas as pd
```



```

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Import scikit-learn modules
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load dataset
df = pd.read_csv("train.csv")

# --- Data Cleaning (ensure numeric predictors) ---
df['Age'] = df['Age'].fillna(df['Age'].median())
df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])
df.drop('Cabin', axis=1, inplace=True)
df.drop_duplicates(inplace=True)

# Encode categorical variables into dummy/indicator columns
df = pd.get_dummies(df, columns=['Sex', 'Embarked', 'Pclass'], drop_first=True)

# 1. Select predictors (drop target + irrelevant text columns)
X = df.drop(['Survived', 'Name', 'Ticket'], axis=1)
y = df['Survived']

# 2. Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. Scale features (important for Logistic Regression convergence)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

```

```
# 4. Train Logistic Regression
```

```
model = LogisticRegression(max_iter=1000, solver='lbfgs')
```

```
model.fit(X_train_scaled, y_train)
```

```
# 5. Predict
```

```
y_pred = model.predict(X_test_scaled)
```

```
# 6. Evaluate
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

```
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```